Universidad Internacional de La Rioja

Escuela Superior de Ingeniería y Tecnología

Grado en Ingeniería Informática

Sistema experto para la toma de decisiones estratégicas en League Of Legends

Trabajo fin de estudio presentado por:	Oscar Sánchez García
Director/a:	Claudia Blanca Gonzalez Calleros
Fecha:	08/04/2026
Repositorio del código fuente:	https://github.com/OSZ8/sistema-experto-tfg-unir

Resumen

En este trabajo de fin de grado he creado una aplicación con la función de ayudar a personas en la toma de decisiones en el videojuego League of Legends (LoL), un videojuego con una cantidad de variables y posibilidades inmensas, cada decisión tomada dentro del juego es relevante y podría ser decisiva para obtener la victoria en la partida en muchos casos.

La tecnología empleada para el desarrollo de la aplicación para solventar estos problemas es la siguiente:

- Backend: Python + Flask
- Frontend: HTML + CSS + JavaScript
- Base de Datos: SQLite
- Motor Experto: Sistema de inferencias propio (encadenamiento hacia adelante) con hechos y reglas definidas
- API Externas: Riot Games API (League V4 + Match V5) para extracción de datos reales

En este trabajo se explican las funcionalidades básicas y variables del videojuego para poder tenerlas en cuenta, de esta forma el programa recomiende la mejor decisión lógica buscando aumentar la probabilidad de victoria en la partida mediante una recomendación de personaje en la fase de selección de personaje y recomendaciones de objetos y anotaciones a tener en cuenta para la partida.

Palabras clave: Sistemas expertos, IA explicable (XAI), eSports

Abstract

In this Bachelor's Thesis, I have developed an application designed to assist users in decision-making within the video game League of Legends (LoL). Given the immense number of variables and possibilities in the game, every decision made is relevant and can often be the deciding factor in securing a victory.

The technology stack used to develop the application and address these challenges is as follows:

- Backend: Python + Flask
- Frontend: HTML + CSS + JavaScript
- Database: SQLite
- Expert Engine: A custom inference system (forward chaining) with defined facts and rules
- External APIs: Riot Games API (League V4 + Match V5) for real-time data extraction

This thesis explains the fundamental mechanics and variables of the game to ensure they are properly accounted for. Consequently, the program recommends the most logical decisions to increase the probability of winning, providing champion suggestions during the champion select phase, as well as item recommendations and strategic notes to consider during the match.

Keywords: Expert system, Explainable AI (XAI), eSports

Índice de contenidos

1. Introducción 1
1.1. Motivación 3
1.2. Conceptos básicos y dominio del problema 4
1.2.1. Ontología de etiquetas y roles 6
1.2.2. El “hecho” (fact) y la explicabilidad 7
1.2.3. La fase de selección (draft) y las reglas de producción 7
1.3. Estructura del trabajo 11
2. Contexto y Estado del Arte 12
2.1. Análisis del contexto estratégico y complejidad del dominio 13
2.1.1. La magnitud del problema: El espacio de decisiones 13
2.2. Estado del arte: soluciones actuales y su evolución 14
2.2.1. Análisis de herramientas comerciales: El predominio de la estadística descriptiva 14
2.2.2. Evolución de la IA en los eSports: De la predicción a la explicación 15
2.2.3. El sistema experto como alternativa pedagógica 15
3. Objetivos y metodología de trabajo 16
3.1. Objetivo general 16
3.2. Objetivos específicos 17
3.3. Metodología de trabajo 18
4. Título del capítulo 20
5. Conclusiones y trabajo futuro 21
5.1. Conclusiones del trabajo 21
5.2. Líneas de trabajo futuro 21
Referencias bibliográficas 22
Anexo A. Título del anexo. 23

Índice de acrónimos 24

Índice de figuras

Figura 1. Mapa de la Grieta del Invocador con las líneas (Top, Mid, Bot y Jungla) 5

Figura 2. Interfaz del recomendador pidiendo al usuario insertar los datos de los campeones de la partida actual 8

Figura 3. Interfaz del recomendador tras un usuario haber insertado los personajes de la partida 9

Figura 4. El programa da como resultado su conclusión tras realizar los calculos y recomienda las mejores opciones 10

Índice de tablas

Tabla 1. Ontología de Etiquetas del Sistema Experto 6

1. Introducción

La industria de los videojuegos ha sufrido cambios constantes en los últimos años. Antiguamente solo se percibía a los videojuegos como una fuente de entretenimiento, hoy en día, muchos de los títulos que solamente trataban de cumplir la función de entretener al usuario se han ido profesionalizando con el tiempo, llegando a convertirse en deportes electrónicos o eSports, comparables incluso a los deportes tradicionales. Dentro de este contexto, los juegos del género MOBA (Multiplayer Online Battle Arena), y en especial League of Legends (LoL), destacan como un escenario muy interesante para aplicar conceptos de ingeniería informática y, en particular, de inteligencia artificial y análisis de datos.

Mi idea no es simplemente crear otra herramienta más de consulta, pues en el mercado actual hay varias ofertas que cumplen con mi propósito, pero tienen ciertos problemas individualmente. Lo que realmente busco es analizar cómo los sistemas basados en conocimiento pueden ayudar a poner orden en un entorno donde la información es incompleta, cambiante y bastante compleja. En resumen, trato de poder ordenar un entorno de variables constantes para buscar la mejor solución a problemas que se dan en la partida.

A lo largo de mis estudios y también gracias a mi experiencia personal con League of Legends, he visto algo que se repite bastante: aunque muchos jugadores tienen un nivel mecánico muy alto, les falla la toma de decisiones estratégicas. Esto ocurre sobre todo en momentos clave como la selección de campeones (draft), la cual puede llegar a determinar gran parte del resultado de la partida sin siquiera jugar esta misma. De hecho, estudios sobre predicción en juegos competitivos sugieren que la fase de selección puede condicionar el éxito del encuentro de manera determinante antes de que los jugadores entren al mapa [1] (Yang et al., 2016).

Para ello, he decidido desarrollar un sistema experto que funcione como una especie de asesor estratégico. La idea es que pueda imitar el razonamiento de un analista humano

experto en la materia, pero aprovechando la capacidad de cálculo de un sistema informático, su velocidad y sin las limitaciones típicas como el cansancio o la presión.

Este proyecto no es relevante solo desde el punto de vista del juego. A nivel más teórico, League of Legends plantea un problema muy interesante relacionado con la teoría de juegos aplicada. Algunos estudios señalan que la dificultad no está únicamente en lo que ocurre dentro de la partida, sino también en todo el conocimiento que rodea al juego, lo que se conoce como "metagame" [2] (Mora-Cantallops & Sicart, 2018). Sustento esta idea gracias a que, con el tiempo, la escena competitiva dentro del mundo de los videojuegos se ha profesionalizado y se han estandarizado posiciones de coach (entrenador), analistas de draft, etc., mostrando que a nivel competitivo la toma de decisiones es relevante a los mismos niveles que saber jugar al juego con unas mecánicas profesionales.

Durante el desarrollo, me he encontrado con varios retos técnicos importantes. Por ejemplo, trabajar con datos en tiempo real usando las APIs oficiales de [3] Riot Games (2024), o diseñar un sistema en Python capaz de no solo dar recomendaciones, sino también explicar por qué las da. Esto último es especialmente relevante, ya que muchos modelos actuales de inteligencia artificial funcionan como "cajas negras" y no ofrecen una justificación clara de sus decisiones, lo que dificulta que el usuario confíe plenamente en el sistema [4] (Russell & Norvig, 2021).

De hecho, he optado por alejarme un poco del enfoque basado únicamente en Machine Learning, que es el más utilizado hoy en día. Aunque estos modelos son muy buenos prediciendo resultados, suelen fallar en algo clave: la explicabilidad. En este trabajo he intentado priorizar precisamente eso, construyendo un sistema que no solo indique qué hacer, sino que también explique el motivo, basándose en factores como las sinergias entre campeones, los enfrentamientos directos o el equilibrio del equipo y sus necesidades.

Por último, esta introducción sirve como punto de partida para entender el resto del documento. En ella explico tanto la motivación detrás del proyecto como los conceptos

básicos necesarios para seguirlo, incluso si no se tiene experiencia previa con este tipo de juegos competitivos. También presento la estructura del trabajo, que dará paso a un análisis más técnico donde se detallará cómo se ha desarrollado el sistema. En esencia, este proyecto intenta dar forma y sentido al caos estratégico de una partida de League of Legends, utilizando herramientas propias de la lógica computacional.

1.1. Motivación

La elección de este tema para mi Trabajo de Fin de Grado no es simplemente un gusto personal por el videojuego y lo que ofrece, también reside en que a nivel regional yo he llegado a competir y soy experto en el tema, por eso mismamente pensé que como experto y jugador podría aportar algo de valor al desarrollo del proyecto y no solamente como ingeniero. Como estudiante y seguidor de la escena competitiva de League of Legends desde hace años, siempre me ha fascinado la enorme brecha que existe entre la ejecución técnica y la estrategia pura. He visto cómo, muy frecuentemente, partidas con jugadores brillantes se perdían en apenas diez minutos simplemente por una mala planificación inicial o por una mala composición del equipo a lo que la partida requería, y yo personalmente he vivido esto muchísimas ocasiones.

Mi principal motivación personal nace de esa frustración, la sensación de que, en un juego tan complejo, la mayoría de los jugadores tomamos decisiones "de memoria" o siguiendo modas, sin entender realmente la lógica que hay detrás. Al investigar las herramientas de asistencia que existen hoy en día, me di cuenta de que casi todas se limitan a darte un porcentaje de victoria o una lista de objetos recomendados basada en estadísticas masivas. Sin embargo, ninguna de ellas te explica el "porqué". Como futuro ingeniero, me motivaba el reto de crear algo diferente, un sistema que no solo te diga qué elegir, sino que sea capaz de razonar su decisión como lo haría un analista profesional.

En cuanto a el plano tecnológico, me planteé el reto de ir un poco a contracorriente de las tendencias actuales. Hoy en día parece que cualquier solución de inteligencia artificial debe pasar obligatoriamente por el Machine Learning, pero estos modelos a menudo funcionan

como cajas negras donde es imposible entender el porqué de un resultado. Mi intención con este trabajo ha sido reivindicar la utilidad de los Sistemas Expertos en entornos estratégicos. Me motivaba profundamente el desafío de programar desde cero un motor de inferencia en Python que fuera capaz de procesar reglas lógicas creadas por humanos. Al final, lo que buscaba era que el usuario no tuviera que creerse la recomendación porque sí. Me parecía mucho más interesante diseñar una arquitectura donde la lógica fuera totalmente abierta, que el programa no se limite a dar una respuesta, sino que sea capaz de explicarle al jugador el porqué de cada decisión de una forma que tenga sentido y sea fácil de seguir. Esto aporta muchísimo valor para los nuevos jugadores que entran por primera vez al juego, pues en mi caso mismamente, tengo amigos que empiezan a jugar y al haber tantas posibilidades y variables se abruman, y la mayoría de las aplicaciones existentes no te explican nada, solamente te dicen lo que tienes que hacer y que tienes que seleccionar y ya.

Otro gran motor de este proyecto ha sido el reto de la implementación real. No quería que este trabajo se quedara en un simple estudio teórico. Me propuse el objetivo de desarrollar una aplicación funcional, utilizando Flask para crear una interfaz web que fuera intuitiva y profesional. Enfrentarme a problemas reales, como la gestión de las APIs de Riot Games, la actualización constante de los datos del juego o el diseño de un sistema de degradación elegante para que la interfaz no falle si una imagen no carga, ha sido una de las partes más enriquecedoras del proceso.

Por concluir, lo que me motiva con este trabajo es la posibilidad de unir mi formación técnica con una de mis pasiones y experiencias competitivas, creando una herramienta que aporte valor real y que demuestre que la inteligencia artificial, cuando se aplica con una base de conocimiento sólida y estructurada, puede ser el mejor aliado para la toma de decisiones en entornos tan caóticos y competitivos como son los eSports.

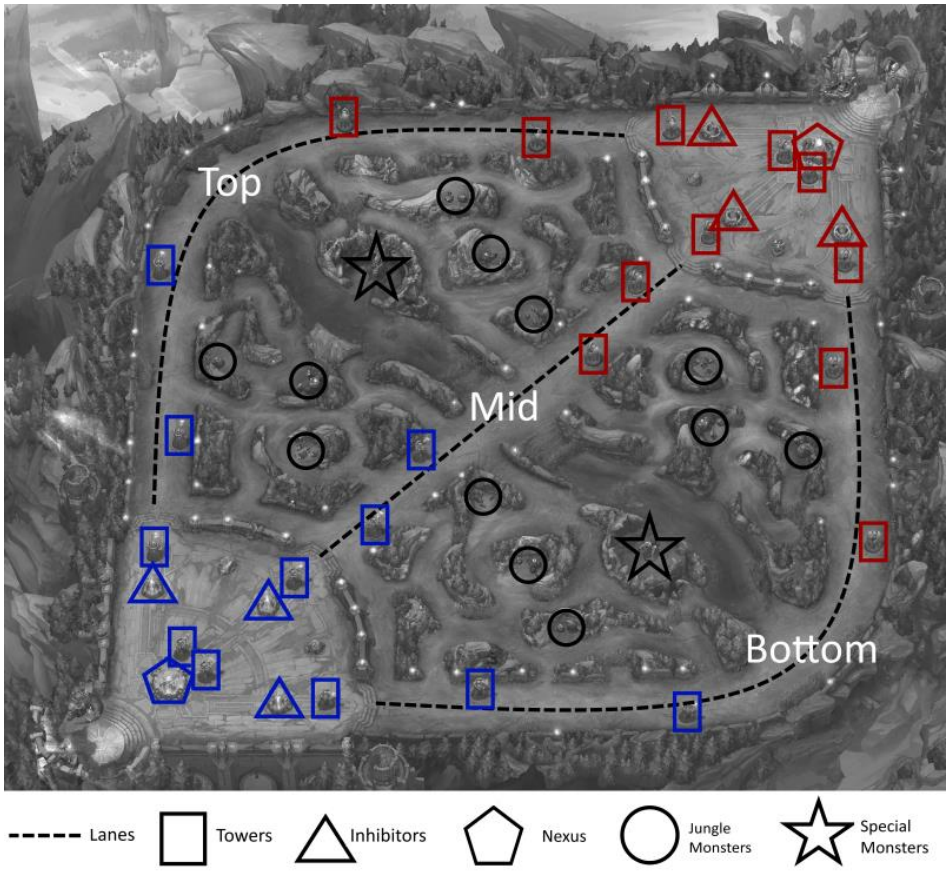
1.2. Conceptos básicos y dominio del problema

Para entender cómo funciona el sistema experto que he desarrollado, primero hay que explicar qué es lo que ocurre realmente en una partida de League of Legends y cómo he

traducido ese caos a un lenguaje que mi programa pueda entender. No se trata solo de un juego de destruir la base enemiga, es un entorno de alta estrategia donde cada elección tiene una consecuencia lógica.

El LoL es un juego de género MOBA (Multiplayer Online Battle Arena) donde dos equipos de cinco jugadores compiten en un mapa cerrado llamado "La Grieta del Invocador". El objetivo final es destruir la base enemiga, también conocida como el Nexo, pero para llegar a ese punto hay que superar una serie de obstáculos y tomar cientos de decisiones tácticas.

Figura 1. *Mapa de la Grieta del Invocador con las líneas (Top, Mid, Bot y Jungla)*



Fuente: [5] League of Legends map and important locations, Afonso et al. (2019)

Como se ve en el mapa, cada jugador ocupa una posición. Sin embargo, lo que realmente importa para mi sistema no es solo dónde se coloca el jugador, sino qué "tipo" de entidad está

manejando. Aquí es donde entra la parte técnica de mi trabajo: la creación de una ontología de etiquetas propia.

1.2.1. Ontología de etiquetas y roles

A diferencia de otras herramientas que usan las categorías estándar que vienen en la documentación oficial, yo he decidido crear un vocabulario de etiquetas propio en mi base de conocimientos. Esto me permite ser mucho más preciso. Por ejemplo, en lugar de decir simplemente que un campeón es un "Mago", mi sistema puede identificar si es un magic_damage (fuente de daño) o si tiene cc_heavy (mucho control de masas).

En la siguiente tabla detallo algunas de las etiquetas clave que he definido en el archivo champions.json y que el motor de inferencia utiliza para razonar:

Tabla 1. Ontología de Etiquetas del Sistema Experto

Etiqueta (Tag)	Significado Estratégico	Función en el Motor
Bruiser / Fighter	Luchador cuerpo a cuerpo.	Evalúa la resistencia en peleas largas.
Marksman	Tirador de daño constante.	Detecta la necesidad de protección o de armadura enemiga.
Healing_self	Autocuración propia.	Dispara la regla de recomendación de "anti-curación".
Cc_heavy	Control de masas múltiple.	Activa la recomendación de "tenacidad" o resistencia.
Magic_damage	Daño de tipo mágico.	Se usa para equilibrar el daño del equipo aliado (AP vs AD).

Fuente: Elaboración propia basada en la arquitectura del motor de inferencia.

1.2.2. El “hecho” (fact) y la explicabilidad

Para que el sistema tome una decisión, necesita convertir lo que pasa en la pantalla en "hechos" lógicos. He diseñado una clase llamada Fact que es el corazón de la inteligencia del programa. Lo que hace especial a esta clase es que no solo guarda el dato (por ejemplo, "hay un enemigo tanque"), sino que guarda el source (la fuente).

Esto es fundamental para lo que comentaba antes sobre la explicabilidad. Como señalan [4] Russell y Norvig (2021), un sistema inteligente es mucho más útil si puede justificar sus conclusiones. Gracias a este atributo source, mi programa no te dice simplemente "compra este objeto", sino que es capaz de decirte "te recomiendo este objeto porque el campeón X tiene la etiqueta de curación".

1.2.3. La fase de selección (draft) y las reglas de producción

El momento donde el sistema realmente demuestra su utilidad es en el Draft. Como he mencionado anteriormente, esta es la fase previa a la partida donde se eligen los personajes y se establecen las bases de la victoria. Es, en estado puro, un juego de "piedra, papel o tijera" pero elevado a una potencia de miles de variables. Como bien señalan [2] Mora-Cantallops y Sicart (2018), el conocimiento profundo del "metagame" y la capacidad de entender las sinergias en esta fase es lo que marca la diferencia entre un jugador promedio y uno experto.

Para que el sistema pueda razonar sobre este escenario, he diseñado una interfaz en Flask que permite al usuario volcar el estado actual de la partida. En términos de ingeniería de inteligencia artificial, lo que estamos haciendo aquí es alimentar la memoria de trabajo del motor de inferencia con "hechos" reales.

Figura 2. *Interfaz del recomendador pidiendo al usuario insertar los datos de los campeones de la partida actual.*

Una vez que el usuario introduce los personajes que están presentes en la sesión, el programa no se limita a leer nombres, lo que hace es mapear esos campeones con las etiquetas de mi base de conocimientos. Por ejemplo, si el usuario introduce a "Soraka", el sistema genera automáticamente un hecho (Fact) asociado a la etiqueta healing_support. A partir de ahí, el motor de inferencia procesa esta información teniendo en cuenta la posición que le falta al usuario, las carencias de su propio equipo y, sobre todo, las amenazas que presenta el equipo enemigo.

Figura 3. *Interfaz del recomendador tras un usuario haber insertado los personajes de la partida.*

Al procesar esta composición de ejemplo, el programa ejecuta sus reglas de producción de forma secuencial. El resultado no es una respuesta única y cerrada, sino una conclusión razonada. El sistema recomienda tres posibles opciones de campeón para seleccionar, pero lo más importante desde el punto de vista de la ingeniería es que justifica su decisión. El programa avisa al usuario en todo momento de qué reglas han saltado y por qué se proponen esas soluciones, cumpliendo así con el principio de explicabilidad que mencionaba en la introducción.

Ilustración 4. *El programa da como resultado su conclusión tras realizar los calculos y recomienda las mejores opciones.*

Como se puede observar en la Figura 4, el sistema realiza un análisis heurístico y muestra qué reglas lógicas han disparado esas recomendaciones. En este caso concreto, las tres reglas principales que han entrado en juego han sido: Anti-Curaciones, Anti-Tanques y Resistencia Mágica.

A la derecha de cada regla, el programa utiliza el atributo source que he implementado en la clase Fact para explicar qué personajes han provocado esa alerta. Por ejemplo, si un personaje como Soraka está presente, el motor detecta su alta capacidad de curación y recomienda automáticamente objetos con la etiqueta antiheal, como el Morellonomicon.

De igual forma ocurre con la regla de Anti-Tanques. Si el sistema identifica personajes robustos como Olaf, recomienda objetos de penetración de armadura. En el ejemplo visual, sugiere el Recordatorio Letal como la opción óptima, ya que es un objeto "híbrido" en su utilidad: aporta la penetración necesaria contra tanques y, a la vez, aplica anti-curaciones, optimizando así el inventario del jugador.

Por último, es relevante destacar que el sistema es capaz de trabajar con información incompleta. Si dejamos la selección del tirador (ADC/Bot) vacía, el programa analiza el hueco restante y propone campeones específicos para esa posición. Para refinar este ranking de tres opciones, el motor combina la lógica de las reglas con un cálculo matemático que cruza el Win Rate individual, el enfrentamiento directo (matchup) contra el ADC enemigo y la sinergia con el resto de los aliados ya seleccionados. De esta forma, paso de una simple lista de personajes a una asesoría estratégica integral basada en evidencias lógicas y estadísticas.

1.3. Estructura del trabajo

Para cerrar esta introducción, quiero resumir brevemente cómo he organizado el resto de la memoria. He dividido el trabajo en seis capítulos que recorren todo el camino, desde que surge la idea y estudio el problema hasta que consigo una solución técnica que funciona. Esta es la hoja de ruta que voy a seguir:

En el Capítulo 2: Contexto y Estado del Arte, me centro en analizar qué se ha hecho hasta ahora en el mundo de la IA aplicada a videojuegos. Aquí es donde comparo mi propuesta de sistema experto con las herramientas que ya existen en el mercado y con otros estudios realizados respecto al mismo tema, intentando explicar por qué mi enfoque en la "explicabilidad" aporta algo nuevo.

En el Capítulo 3: Objetivos y Metodología, detallo qué quiero conseguir exactamente con este proyecto (objetivos SMART) y qué pasos voy a seguir para no perderme por el camino. Explico cómo he usado Python y Flask siguiendo un orden lógico de desarrollo de software.

En el Capítulo 4: Análisis y Diseño de la Solución, entro de lleno en la arquitectura del sistema. En este apartado detallo los requisitos funcionales y el diseño de los componentes clave, como la base de conocimientos y el motor de inferencia. Es aquí donde explico formalmente cómo he modelado los "hechos" y las "reglas" para que el programa sea capaz de procesar el conocimiento experto que he inyectado en el sistema.

En el Capítulo 5: Implementación y Pruebas, describo el proceso de desarrollo técnico y la integración de las distintas tecnologías. Detallo el uso de Python para la lógica de negocio, Flask para la capa web y la conexión con la API de Riot Games. Además, presento los protocolos de prueba y los resultados obtenidos al someter al recomendador a situaciones de partida reales, validando así su precisión.

Finalmente, en el Capítulo 6: Conclusiones y Trabajo Futuro, realizo una síntesis de los resultados y valoro el grado de cumplimiento de los objetivos planteados. Este capítulo sirve también para proponer posibles expansiones del proyecto, como la implementación de lógica difusa o la automatización de la captura de datos en tiempo real, dejando la puerta abierta a futuras evoluciones de la herramienta.

2. Contexto y Estado del Arte

En este capítulo voy a profundizar en el marco teórico y tecnológico que rodea a este proyecto. Para entender por qué es necesario un sistema experto en League of Legends, primero es fundamental analizar la magnitud del problema estratégico al que se enfrentan los jugadores y, posteriormente, revisar qué soluciones se han desarrollado hasta la fecha, tanto en el ámbito académico como en el comercial. Mi objetivo aquí es situar mi propuesta dentro de la evolución de la Inteligencia Artificial, justificando por qué he decidido apostar por un modelo basado en reglas en un momento donde el Machine Learning parece dominarlo todo.

2.1. Análisis del contexto estratégico y complejidad del dominio

Para cualquier observador externo, un videojuego puede parecer un entorno simple de estímulos y diversión. Sin embargo, desde el punto de vista de la ingeniería de software, League of Legends es un sistema de una complejidad técnica abrumadora. Nos encontramos ante un entorno de información imperfecta y un espacio de estados que crece exponencialmente con cada nueva variable que se añade al juego.

2.1.1. La magnitud del problema: El espacio de decisiones

La complejidad del draft no es una exageración subjetiva. Si consideramos que hay más de 170 campeones disponibles y que en cada partida se seleccionan 10 (sin contar los 10 bloqueos iniciales ya que cada persona en la fase de draft tiene opción a prohibir el uso de un personaje en la partida), el número de combinaciones posibles es astronómico. A esto hay que sumar que cada campeón tiene un set de habilidades único y estadísticas que cambian en cada parche. Según [6] Hodge et al. (2017), los juegos de tipo MOBA presentan retos de optimización y predicción que superan en muchas dimensiones a juegos clásicos, debido a la naturaleza dinámica de los datos y a que la ejecución se realiza en tiempo real con información incompleta.

He observado que esta saturación de información genera lo que en psicología cognitiva se conoce como "carga cognitiva excesiva". El jugador, bajo la presión del cronómetro en la fase de selección, no puede procesar todos los enfrentamientos, sinergias y debilidades de forma puramente racional. Es en este punto donde he identificado la necesidad de una herramienta que actúe como un filtro inteligente, transformando miles de datos brutos en recomendaciones digeribles que permitan al usuario tomar la mejor decisión posible sin verse desbordado por la complejidad del sistema.

2.2. Estado del arte: soluciones actuales y su evolución

Para situar mi proyecto en el panorama tecnológico actual, es necesario analizar qué herramientas tienen a su disposición los jugadores hoy en día y cómo ha evolucionado la investigación académica en este campo. A grandes rasgos, podemos dividir las soluciones existentes en dos categorías, las plataformas comerciales basadas en analítica de datos masivos y los estudios académicos centrados en modelos predictivos de inteligencia artificial.

2.2.1. Análisis de herramientas comerciales: El predominio de la estadística descriptiva

En el ecosistema competitivo de League of Legends, han proliferado herramientas muy populares como OP.GG, Blitz.gg, Porofessor, Mobalytics o U.GG. Tras analizar el funcionamiento de estas plataformas, he identificado que su principal fortaleza es, al mismo tiempo, su mayor limitación: se basan casi exclusivamente en la analítica descriptiva.

Estas herramientas recolectan millones de datos de partidas a través de la API de Riot Games para mostrar tendencias. Por ejemplo, pueden indicarte que un campeón tiene un 53% de win rate o que un objeto es el más frecuente en una determinada posición. Sin embargo, como mencionan [4] Russell y Norvig (2021) en su análisis sobre la utilidad de los sistemas inteligentes, la información sin contexto tiene un valor limitado para la toma de decisiones complejas. El problema de estas aplicaciones es que funcionan como "cajas negras", te dicen qué elegir, pero no te explican por qué esa elección es buena contra tu rival específico o cómo encaja en la sinergia de tu equipo.

Lo que busco con mi sistema experto es alejarme de esa estadística fría. Mientras que Blitz.gg te recomienda un objeto porque "mucha gente lo compra", mi motor de inferencia te lo recomienda porque ha detectado un "hecho" lógico (por ejemplo, el enemigo tiene la etiqueta healing_self) y ha aplicado una regla de conocimiento experto.

2.2.2. Evolución de la IA en los eSports: De la predicción a la explicación

En el ámbito académico, la investigación ha avanzado significativamente hacia la predicción de resultados. Un estudio fundamental en este campo es el de [6] Hodge et al. (2017), quienes exploraron cómo algoritmos de Machine Learning pueden predecir quién ganará una partida de un MOBA analizando las variables del draft. Estos modelos son extremadamente precisos, pero suelen requerir una capacidad computacional enorme y son difíciles de interpretar para un usuario que no sea experto en datos.

Recientemente, el foco de la comunidad científica ha empezado a girar hacia la Inteligencia Artificial Explicable (XAI). Según [7] Adadi y Berrada (2018), el valor real de una IA en entornos donde hay un humano tomando la decisión final (como es el caso de un jugador en plena fase de selección) reside en su capacidad para justificar sus conclusiones. No basta con que el sistema sea un oráculo que acierta, el usuario necesita entender el razonamiento para confiar en la herramienta y, de paso, mejorar su propio criterio estratégico. Es precisamente en esta corriente de la XAI donde se encuadra mi propuesta de Sistema Experto.

2.2.3. El sistema experto como alternativa pedagógica

A diferencia de los modelos de aprendizaje profundo (Deep Learning) que se han usado en proyectos famosos como OpenAI Five para Dota 2, yo he apostado por un sistema basado en reglas de producción. La razón es sencilla, la flexibilidad y la pedagogía.

En un entorno tan cambiante como el de League of Legends, donde cada dos semanas hay un parche que cambia las reglas del juego, un sistema experto es mucho más ágil. No necesito reentrenar un modelo con millones de nuevas partidas, simplemente puedo actualizar la base de conocimientos o ajustar una regla de prioridad. Esta capacidad de inyectar conocimiento experto de forma inmediata es lo que permite que mi solución sea más que un simple recomendador, es un consultor lógico que ayuda al jugador a comprender la capa interna estratégica del juego.

3. Objetivos y metodología de trabajo

Tras haber analizado en los capítulos anteriores la enorme complejidad técnica que rodea a League of Legends y comprobar cómo las soluciones actuales suelen dejar al usuario a ciegas respecto a la lógica de sus consejos, en este apartado voy a concretar qué es lo que me he propuesto lograr con este proyecto. No se trata solo de programar una herramienta que funcione, sino de seguir un proceso estructurado que garantice que el sistema experto es fiable, escalable y, sobre todo, útil para un jugador que busca mejorar su capacidad estratégica.

En las siguientes páginas, voy a desglosar mi objetivo general en varias metas más pequeñas y manejables, lo que me permitirá avanzar con paso firme durante el desarrollo. Además, describiré la metodología de trabajo que he seguido, un camino que combina la disciplina de la ingeniería de software con la flexibilidad necesaria para adaptar el conocimiento de un analista humano a un motor de reglas en Python. Como bien indica [8] Sommerville (2011), cualquier proyecto de software exitoso debe basarse en un proceso bien definido que permita gestionar la complejidad, y en mi caso, ese proceso ha sido fundamental para no perderme entre los más de 170 campeones y las miles de interacciones posibles del juego.

3.1. Objetivo general

El objetivo principal de este trabajo es diseñar e implementar un sistema experto basado en reglas que asista a los jugadores de League of Legends durante la fase de draft, proporcionando recomendaciones estratégicas de personajes y equipamiento que sean totalmente explicables y fáciles de comprender para el usuario, haciendo que este no solo tome la mejor decisión táctica a nivel de numero en porcentaje de victorias, sino que también llegue a entender el motivo detrás de esta decisión.

Con este proyecto, no busco simplemente crear otro recomendador estadístico basado en números brutos, sino desarrollar una herramienta que sea capaz de razonar sus propuestas. La idea es que el sistema actúe como un consultor táctico, traduciendo el conocimiento

experto sobre sinergias y debilidades del juego en consejos directos. Para lograrlo, he apostado por construir un motor de inferencia propio en Python y una interfaz web accesible mediante Flask, permitiendo que cualquier jugador pueda recibir una asesoría estratégica de alto nivel en tiempo real.

3.2. Objetivos específicos

Para alcanzar la meta principal que acabo de describir, he desglosado el trabajo en los siguientes objetivos operativos:

- **Diseñar y estructurar una base de conocimientos propia:** No se trata solo de volcar datos de la API de Riot Games, sino de categorizar a los más de 170 campeones mediante una ontología de etiquetas (tags) que represente fielmente sus roles estratégicos, tipos de daño y utilidades específicas.
- **Desarrollar el motor de inferencia en Python:** Programar la lógica central del sistema capaz de procesar reglas de producción tipo “IF-THEN”. Este motor debe ser capaz de analizar la memoria de trabajo (los campeones elegidos en la partida) y disparar las recomendaciones pertinentes.
- **Implementar un sistema de explicabilidad (XAI):** Asegurar que el programa no sea una caja negra. El objetivo es que el sistema guarde y muestre la traza del razonamiento, indicando al usuario exactamente qué etiquetas y qué reglas han motivado cada sugerencia de objeto o campeón.
- **Construir una interfaz web funcional con Flask:** Crear una plataforma accesible e intuitiva donde el usuario pueda introducir la composición de los equipos en tiempo real y recibir las recomendaciones de forma visual y rápida.
- **Validar la precisión del sistema mediante escenarios reales:** Someter al sistema experto a pruebas con composiciones de equipo reales para comprobar que las reglas de contra-elección (counter-pick) y sinergias funcionan correctamente y ofrecen una ventaja táctica real.

3.3. Metodología de trabajo

Para el desarrollo de este proyecto, no he seguido un proceso lineal y rígido, sino que he apostado por una metodología iterativa e incremental. Esto me ha permitido construir el sistema por módulos, validando cada avance antes de pasar al siguiente. De esta forma, he podido ajustar la lógica del motor de inferencia a medida que descubría nuevas interacciones entre los campeones durante las pruebas.

El trabajo se ha estructurado en las siguientes cuatro fases principales:

- **1. Ingeniería del conocimiento y extracción de datos**

La primera etapa consistió en definir de dónde vendría la inteligencia del sistema. Aquí realicé una investigación profunda sobre el metagame actual y utilicé la API de Riot Games (DataDragon) para obtener los datos base. El reto en esta fase no fue solo descargar información, sino transformarla en algo útil, creé una ontología de etiquetas propia en un archivo JSON que permitiera al sistema clasificar a los personajes por su comportamiento real en la partida (como healing_self o cc_heavy) y no solo por su nombre.

- **2. Diseño y desarrollo del motor de inferencia**

Con los datos ya estructurados, pasé a programar el cerebro de la aplicación en Python. Siguiendo los principios de los sistemas expertos, diseñé la clase Fact para gestionar la memoria de trabajo y la clase Engine para procesar las reglas de producción. En esta fase, el trabajo fue puramente de lógica, traducir el conocimiento estratégico de un jugador experto a reglas de programación tipo IF-THEN.

- **3. Integración de la Interfaz Web y Experiencia de Usuario**

Una vez que el motor era capaz de dar recomendaciones correctas por consola, desarrollé la capa visual utilizando el framework Flask. El objetivo aquí fue crear una interfaz que fuera rápida de usar (crucial durante un draft real) y que permitiera mostrar visualmente la traza

del razonamiento. Esta fase incluyó la gestión de recursos estáticos, como las imágenes de los campeones.

- **4. Pruebas, Validación y Refinamiento**

La última fase ha sido un proceso constante de "prueba y error" utilizando casos reales de partidas clasificatorias y de la escena profesional. He sometido al sistema a composiciones de equipo variadas para verificar si las recomendaciones eran coherentes o se ajustaban a casos reales. Esto me permitió ajustar las prioridades de las reglas (por ejemplo, dando más peso a la falta de daño mágico o a la necesidad de cortar curaciones enemigas) hasta lograr un equilibrio que aporte un valor estratégico real al usuario.

4. Título del capítulo

5. Conclusiones y trabajo futuro

5.1. Líneas de trabajo futuro

Referencias bibliográficas

[1] Yang, P., Harrison, B., & Roberts, D. L. (2016). Predicting Winning Teams in Dota 2. In Proceedings of the 11th International Conference on the Foundations of Digital Games.

[2] Mora-Cantallops, M., & Sicart, M. (2018). League of Legends, out of the box: A video game's impact on the esports ecosystem. Entertainment Computing, 26, 1-9.

[3] Riot Games. (2024). Data Dragon Tool & API Documentation. Riot Developer Portal.

[4] Russell, S. J., & Norvig, P. (2021). Artificial Intelligence: A Modern Approach (4th ed.). Pearson.

[5] Ana Paula Afonso, Maria Beatriz Carmo, Tiago Gonçalves, Pedro Vieira (2019) VisuaLeague: Player performance analysis using spatial-temporal data. Researchgate.

[6] Hodge, V. J., Devlin, S., Sephton, N., Block, F., Drachen, A., & Cowling, P. I. (2017). Win Prediction in Multi-player Online Battle Arenas. IEEE Transactions on Games, 9(2), 115-126.

[7] Adadi, A., & Berrada, M. (2018). Peeking Inside the Black-Box: A Survey on Explainable Artificial Intelligence (XAI). IEEE Access, 6, 52138-52160.

[8] Sommerville, I. (2011). Ingeniería de software (9.ª ed.). Pearson Educación.

Anexo A. Título del anexo.

Índice de acrónimos